Disk and File System Analysis

DF: Disk and File System Forensics

Analyze and Analysis

NSSECU3 | Digital Forensics

Version	1.0
Date of Version	2024 May 14
Created by	Jerome Gutierrez
Confidentiality Level	Internal

Overview

1. Media Analysis
2. File System Abstraction Model
3. File Slack
4. Analyzing Image Files
5. Forensic Imaging
6. Hashing
7. Carving

Media Analysis Concepts

Identification → Extraction → Analysis

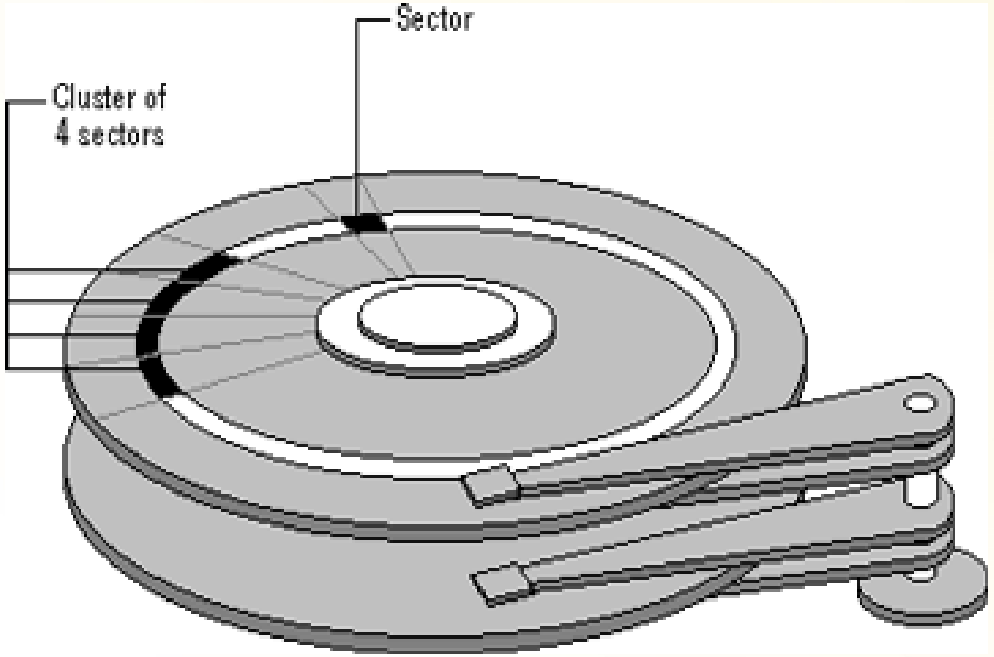
Media Analysis Concepts

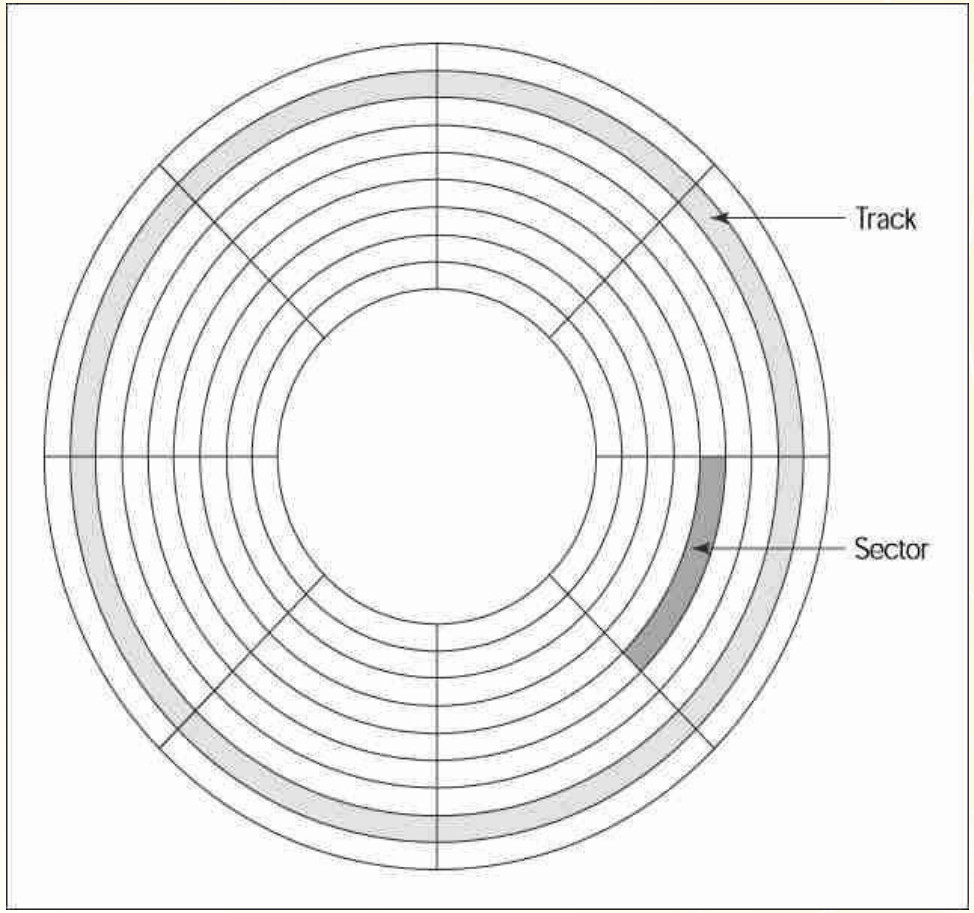
Acquisition	Analysis	Presentation
Identification	**Extraction**	**Analysis**
• determining which active and deleted files are available in a volume	• retrieving of relevant file data and metadata	• applying our intelligence to the data set and ideally produce meaningful results

File System Abstraction Model

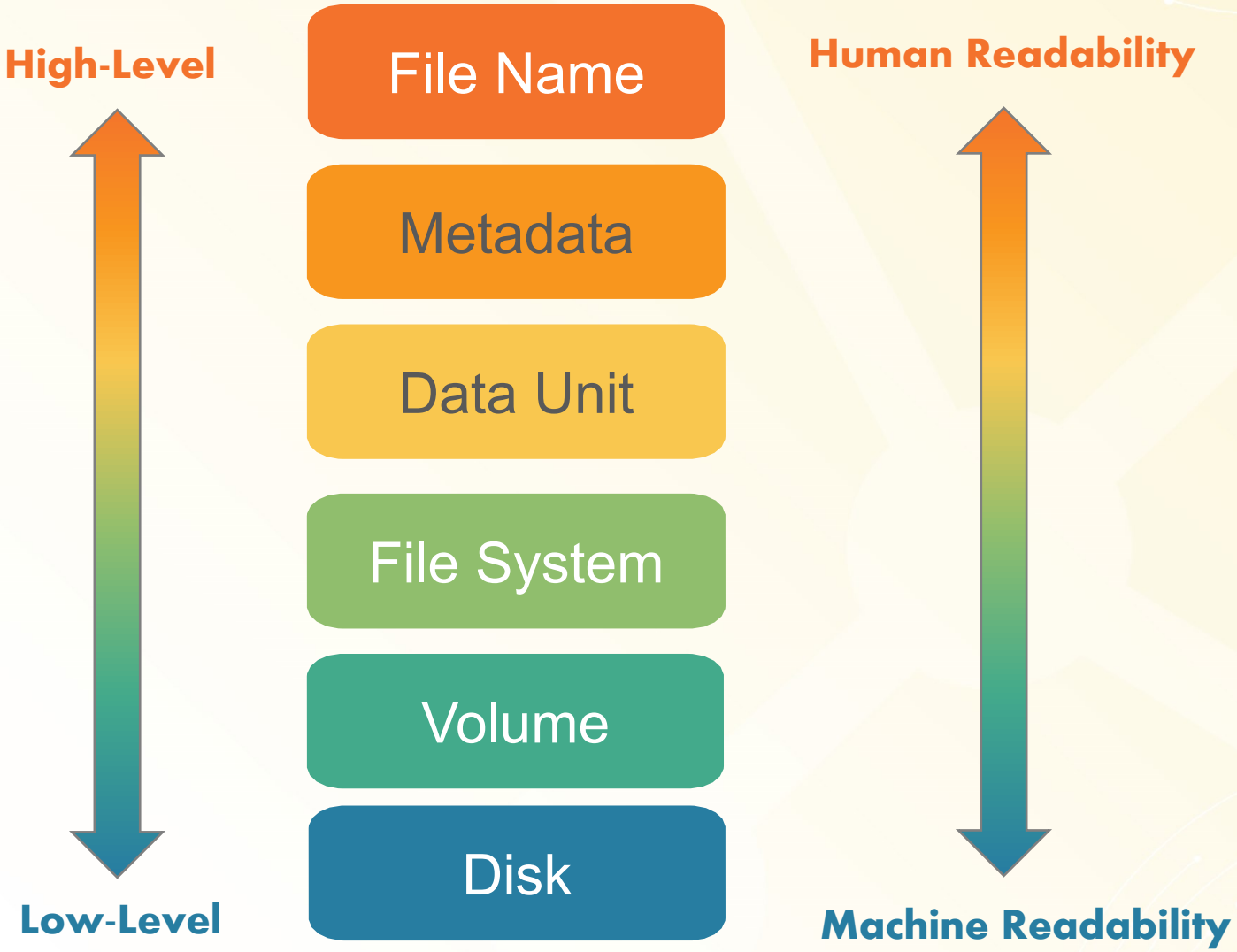
Disk → Volume → File System → Data Unit → Metadata → File Name

File System Forensics: HDD





File System Abstraction Model (FSAM)



File System Abstraction Model (FSAM)

Low-Level

Disk	• **physical** storage device
Volume	• created using **all or part of** one or more disks
File System	• laid down on a volume and describes the layout of files and their associated metadata
Data Unit	• smallest available freestanding unit of data storage available in a given file system • also known as blocks (Linux / Unix) or clusters (Windows)
Metadata	• data about data • also known as **inodes**
File Name	• consists of file and folder/directory names

High-Level

File System Abstraction Model

	sample.jpg	INODE2
	prog.c	INODE0
	check.pdf	INODE5

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK10, BLK13, BLK14, BLK21
INODE 3	
INODE 4	
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

File Name

Metadata

Data Unit

Deleted Data

Categories of Deleted Data

Just because **data is not visible** from the file listing doesn't mean that it's not on the drive

Deleted files → Orphaned Files → Unallocated Files → Overwritten files

Deleted Files

Deleted files → Orphaned Files → Unallocated Files → Overwritten files

- Are the **"most recoverable"**
- Generally, refers to files that have been *"unlinked"*
  - the filename is <u>no longer presented</u> when a *user views a directory*,
  - the <u>filename, metadata, structure, and data units</u> are marked as *"free"*
- The connections between these layers are still intact when forensic techniques are applied to the file system
- **Recovery** consist of recording the relevant filename and metadata structures and then extracting the data units

Deleted Files

Deleted files → Orphaned Files → Unallocated Files → Overwritten files

✕	sample.jpg	INODE2
	prog.c	INODE0
	check.pdf	INODE5

File Name

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK10, BLK13, BLK14, BLK21
INODE 3	
INODE 4	
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	

Metadata

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

Data Unit

Deleted Files

Deleted files → Orphaned Files → Unallocated Files → Overwritten files

sample.jpg
is marked as deleted

✕	sample.jpg	INODE2
	prog.c	INODE0
	check.pdf	INODE5

File Name

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK10, BLK13, BLK14, BLK21
INODE 3	
INODE 4	
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	

Metadata

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

Data Unit

Orphaned Files

Deleted files → **Orphaned Files** → Unallocated Files → Overwritten files

- Are similar to deleted files except the link between the filename and metadata structure is <u>no longer accurate</u>

- **Recovery of data** (and metadata structure) is still possible but there is <u>no direct correlation</u> from the file name to recovered data

Orphaned Files

Deleted files → **Orphaned Files** → Unallocated Files → Overwritten files

✕	sample.jpg	INODE2
	prog.c	INODE0
	check.pdf	INODE5

File Name

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK10, BLK13, BLK14, BLK21
INODE 3	
INODE 4	
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	

Metadata

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

Data Unit

Orphaned Files

Deleted files → **Orphaned Files** → Unallocated Files → Overwritten files

Allocation for filename of sample.jpg is used by work.ps

	work.ps	INODE4
	prog.c	INODE0
	check.pdf	INODE5

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK10, BLK13, BLK14, BLK21
INODE 3	
INODE 4	BLK16, BLK24, BLK25
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

File Name

Metadata

Data Unit

Unallocated Files

Deleted files → Orphaned Files → **Unallocated Files** → Overwritten files

- Are data that had their once-allocated file name entry and associated metadata structure become unlinked and/or reused
- Path to **recovery** is through **carving** the <u>not-yet-reused data units</u> from the *unallocated space* of the volume

Unallocated Files

Deleted files → Orphaned Files → **Unallocated Files** → Overwritten files

	work.ps	INODE4
	prog.c	INODE0
	check.pdf	INODE5

File Name

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK10, BLK13, BLK14, BLK21
INODE 3	
INODE 4	BLK16, BLK24, BLK25
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	

Metadata

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

Data Unit

Unallocated Files

Deleted files → Orphaned Files → **Unallocated Files** → Overwritten files

Metadata for sample.jpg is reused by docu.zip

	work.ps	INODE4
	docu.zip	INODE2
	prog.c	INODE0
	check.pdf	INODE5

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK07, BLK11, BLK18
INODE 3	
INODE 4	BLK16, BLK24, BLK25
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

File Name | Metadata | Data Unit

Overwritten Files

Deleted files → Orphaned Files → Unallocated Files → **Overwritten files**

- Are types of data that have had one or more of their data units reallocated to another file
- **Full recovery is no longer possible**, but *partial recovery* may depend on the extent of overwriting
- Data with filenames and/or metadata structures intact that have had some or all data units overwritten are sometimes referred to as **Deleted/Overwritten or Deleted/Reallocated**

Overwritten Files

Deleted files → Orphaned Files → Unallocated Files → Overwritten files

	work.ps	INODE4
	docu.zip	INODE2
	prog.c	INODE0
	check.pdf	INODE5

File Name

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK07, BLK11, BLK18
INODE 3	
INODE 4	BLK16, BLK24, BLK25
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	

Metadata

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

Data Unit

Overwritten Files

Deleted files → Orphaned Files → Unallocated Files → Overwritten files

Data unit for sample.jpg is reused by sound.avi

	work.ps	INODE4
	docu.zip	INODE2
	prog.c	INODE0
	sound.avi	INODE7
	check.pdf	INODE5

File Name

INODE 0	BLK00, BLK01, BLK04
INODE 1	
INODE 2	BLK07, BLK11, BLK18
INODE 3	
INODE 4	BLK16, BLK24, BLK25
INODE 5	BLK26, BLK27, BLK30, BLK31
INODE 6	
INODE 7	BLK08, BLK09, BLK10

Metadata

BLK00	BLK01	BLK02	BLK03
BLK04	BLK05	BLK06	BLK07
BLK08	BLK09	BLK10	BLK11
BLK12	BLK13	BLK14	BLK15
BLK16	BLK17	BLK18	BLK19
BLK20	BLK21	BLK22	BLK23
BLK24	BLK25	BLK26	BLK27
BLK28	BLK29	BLK30	BLK31

Data Unit

File Slack

Also known as Slack Space

File Slack

Assumption: a 4KB (or 4096 bytes) block size on a standard drive with 512-byte sectors

How many sectors per block? 8 sectors per block

Assume a block of data that is part of a file that is recently deleted

C	o	r	y	w	o	r	k	o	r	e	d	D	i	g	i	a	N	_	C	a	l	_	p	_	f	o	r	_	f	o	r
_	a	l	t	e	d	_	a	_	s	e	v	t	a	l	_	a	r	v	e	r	o	v	i	e	n	s	i	e	n	s	i
h	e	i	d	s	_	t	h	e	r	a	l	F	o	r	e	y	_	i	s	d	e	r	_	c	s	_	a	c	s	_	a
_	i	s	_	e	_	s	e	_	p	a	p	n	s	i	c	_	a	_	v	o	f	_	l	n	a	l	y	n	a	l	y
:				:				:				:				:				:				:				:			
a	l	s	o	a	u	t	h	t	h	e	_	H	a	r	l	g	l	o	b	d	i	s	k	h	o	l	d	h	o	l	d

File Slack

Note : Minimum space that can be allocated on a volume is a single block

What will happen if we create an ASCII text file that contains a single byte (e.g., letter A)?

C	o	r	y	w	o	r	k	o	r	e	d	D	i	g	i	a	N	_	C	a	l	_	p	_	f	o	r	_	f	o	r
_	a	l	t	e	d	_	a	_	s	e	v	t	a	l	_	a	r	v	e	r	o	v	i	e	n	s	i	e	n	s	i
h	e	i	d	s	_	t	h	e	r	a	l	F	o	r	e	y	_	i	s	d	e	r	_	c	s	_	a	c	s	_	a
_	i	s	_	e	_	s	e	_	p	a	p	n	s	i	c	_	a	_	v	o	f	_	l	n	a	l	y	n	a	l	y
:				:				:				:				:				:				:				:			
a	l	s	o	a	u	t	h	t	h	e	_	H	a	r	l	g	l	o	b	d	i	s	k	h	o	l	d	h	o	l	d

File Slack

Note : Minimum space that can be allocated on a volume is a single block

What will happen if we create an ASCII text file that contains a single byte (e.g., letter A)?

FIRST… the cluster to be used is marked as “allocated” and assigned to the file’s metadata structure

C	o	r	y	w	o	r	k	o	r	e	d	D	i	g	i	a	N	_	C	a	l	_	p	_	f	o	r	_	f	o	r
_	a	l	t	e	d	_	a	_	s	e	v	t	a	l	_	a	r	v	e	r	o	v	i	e	n	s	i	e	n	s	i
h	e	i	d	s	_	t	h	e	r	a	l	F	o	r	e	y	_	i	s	d	e	r	_	c	s	_	a	c	s	_	a
_	i	s	_	e	_	s	e	_	p	a	p	n	s	i	c	_	a	_	v	o	f	_	l	n	a	l	y	n	a	l	y
:				:				:				:				:				:				:				:			
a	l	s	o	a	u	t	h	t	h	e	_	H	a	r	l	g	l	o	b	d	i	s	k	h	o	l	d	h	o	l	d

File Slack

Note : Minimum space that can be allocated on a volume is a single block

What will happen if we create an ASCII text file that contains a single byte (e.g., letter A)?

SECOND…the letter A, followed by 511 null bytes (0x00) are placed in the first sector

A	0	0	0	w	o	r	k	o	r	e	d	D	i	g	i	a	N	_	C	a	l	_	p	_	f	o	r	_	f	o	r
0	0	0	0	e	d	_	a	_	s	e	v	t	a	l	_	a	r	v	e	r	o	v	i	e	n	s	i	e	n	s	i
0	0	0	0	s	_	t	h	e	r	a	l	F	o	r	e	y	_	i	s	d	e	r	_	c	s	_	a	c	s	_	a
0	0	0	0	e	_	s	e	_	p	a	p	n	s	i	c	_	a	_	v	o	f	_	l	n	a	l	y	n	a	l	y
:				:				:				:				:				:				:				:			
0	0	0	0	a	u	t	h	t	h	e	_	H	a	r	l	g	l	o	b	d	i	s	k	h	o	l	d	h	o	l	d

File Slack

What happens to the other seven sectors?

The original data are **retained**.

This is known as **file slack** or **slack space.**

A	0	0	0	w	o	r	k	o	r	e	d	D	i	g	i	a	N	_	C	a	l	_	p	_	f	o	r	_	f	o	r
0	0	0	0	e	d	_	a	_	s	e	v	t	a	l	_	a	r	v	e	r	o	v	i	e	n	s	i	e	n	s	i
0	0	0	0	s	_	t	h	e	r	a	l	F	o	r	e	y	_	i	s	d	e	r	_	c	s	_	a	c	s	_	a
0	0	0	0	e	_	s	e	_	p	a	p	n	s	i	c	_	a	_	v	o	f	_	l	n	a	l	y	n	a	l	y
:				:				:				:				:				:				:				:			
0	0	0	0	a	u	t	h	t	h	e	_	H	a	r	l	g	l	o	b	d	i	s	k	h	o	l	d	h	o	l	d

Analyzing Image Files

Besides FTK and Autopsy, we have TSK

The Sleuth Kit (TSK)

- A collection of tools for forensic analysis of computer systems

- Used to perform data extraction from disk images of Windows and Unix/Linux systems

FSAM x TSK Commands

Disk	image	img_	
Volume	Media management	mm-	
File System	File System | Journal	fs-	j-
Data Unit	Block	blk-	
Metadata	Inodes	i-	
File Name	File Name	f-	

FSAM x TSK Commands

Disk	image	img_	tools that operate against **image** files
Volume	Media management	mm-	tools that operate on volumes (aka “**m**edia **m**gmt”)
File System	File System | Journal	fs- j-	tools that operate on **f**ile system **s**tructures & **j**ournals
Data Unit	Block	blk-	tools that operate at the data unit (or “**b**lo**c**k”) layer
Metadata	Inodes	i-	tools that operate at the metadata (or “**i**node”) layer
File Name	File Name	f-	tools that operate at the **f**ile name layer

FSAM x TSK Commands

Volume Layer Tools	mm-
File System Layer Tools	fs-
Data Unit Layer Tools	blk-
Metadata Layer Tools	i-
File Name Layer Tools	f-
FS Journal Tools	j-
Image Tools	img_

-stat
- ❑ displays general information about the queried item
- ❑ similar to the “stat” command on Unix-like systems

-ls
- ❑ lists the contents of the queried layer
- ❑ similar to the “ls” command on Unix-like systems

-cat
- ❑ dumps/extracts the content of the queried layer
- ❑ similar to the “cat” command on Unix-like systems

FSAM x TSK Commands

Volume Layer Tools	mm-	-stat	-ls	-cat
File System Layer Tools	fs-	-stat		
Data Unit Layer Tools	blk-	-stat	-ls	-cat
Metadata Layer Tools	i-	-stat	-ls	-cat
File Name Layer Tools	f-		-ls	
FS Journal Tools	j-	-stat	-ls	-cat
Image Tools	img_	-stat		-cat

FSAM x TSK Commands

TSK provides us with more than 25 CLI tools. The tools available are:

1. Media management (Volume): mmls , mmstat , and mmcat
2. File system: fsstat
3. Content: blkcat , blkls , blkstat , and blkalc
4. Metadata: icat , ils , istat , and ifind
5. File name: fls and ffind
6. File system journal: jcat and jls
7. Image File: img_stat and img_cat

Media Management (Volume System) Tools

- **mmstat**

  -Display details about the volume system (partition tables)

- **mmls**

  -Parses and displays the media management structures on the image file or disk (i.e., the partition table)

  -shows non-allocated spaces before, after, or between volumes

```
user@forensics:~$ mmls 10-ntfs-disk.dd
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors
Slot Start End Length Description
00:   Meta    0000000000      0000000000     0000000001   Primary Table (#0)
01:   -----   0000000000      0000000062     0000000063     Unallocated
02:   00:00   0000000063      0000096389     0000096327     NTFS (0x07)
03:   00:01   0000096390      0000192779     0000096390     NTFS (0x07)
04:   -----   0000192780      0000192783     0000000004     Unallocated
```

Media Management (Volume System) Tools

- **mmcat**

-Output the contents of a partition to STDOUT

redirects the output to a file

```
user@forensics:~$ mmls 10-ntfs-disk.dd
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors
Slot Start End Length Description
00:   Meta    0000000000      0000000000     0000000001   Primary Table (#0)
01:   -----   0000000000      0000000062      0000000063      Unallocated
02:   00:00   0000000063      0000096389      0000096327      NTFS (0x07)
03:   00:01   0000096390      0000192779      0000096390      NTFS (0x07)
04:   -----   0000192780      0000192783      0000000004      Unallocated
user@forensics:~$ mmcat 10-ntfs-disk.dd 2> firstntfs.img
:
:
```

File System Tools

- **fsstat**

  - Display general details of a file system

```
user@forensics:~$ fsstat ubnist1.casper-rw.gen3.aff
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: Ext3 Volume
Name:
Volume ID: 9935811771d9768b49417b0b3b881787
Last Written at: Tue Jan 6 10:59:33 2009
Last Checked at: Sun Dec 28 12:37:56 2008
Last Mounted at: Tue Jan 6 10:59:33 2009
Unmounted properly Last
mounted on: Source OS: Linux
Dynamic Structure
Compat Features: Journal, Ext Attributes, Resize Inode, Dir Index
InCompat Features: Filetype, Needs Recovery,
```

Some very useful information

File System Tools

- **fsstat**
  - Display general details of a file system
  - Note: Output varies depending on disk file system

```
user@forensics:~$ fsstat ubnist1.casper-rw.gen3.aff
:
:
METADATA INFORMATION
------------------------------------
Inode Range: 1 - 38401
Root Directory: 2
Free Inodes: 36976
Orphan Inodes: 35, 20, 17, 16,
CONTENT INFORMATION
------------------------------------
Block Range: 0 - 153599
Block Size: 4096
Free Blocks: 85287
```

File System Tools

- **fls**
  - List file and directory names in a disk image (deleted and allocated).

```
user@forensics:~$ fls ubnist1.casper-rw.gen3.aff
d/d 11: lost+found
r/r 12: .wh..wh.aufs
d/d 7681: .wh..wh.plnk
d/d 23041: .wh..wh..tmp
d/d 7682: rofs
d/d 23042: etc
d/d 23044: cdrom
d/d 30721: tmp
d/d 30722: lib
d/d 15377: usr
d/d 7712: sbin
d/d 13: root
r/r * 35(realloc):.aufs.xino
d/d 38401: $OrphanFiles
```

- **ffind**
  - Finds the name of the file or directory using a given INODE

Data Unit Layer Tools

- **blkstat**
  - Display details of a file system data unit (i.e. block or sector)

```
user@forensics:~$ blkstat ubnist1.casper-rw.gen3.aff 521
Fragment: 521
Allocated
```

Data Unit Layer Tools

- **blkcat**

  - Display the contents of file system data unit in a disk image.

  - Stream the content of a given data unit to STDOUT

```
user@forensics:~$ blkcat ubnist1.casper-rw.gen3.aff 521 | xxd | head
0000000:   0200  0000  0c00  0102  2e00  0000  0200  0000  ........
0000010:   0c00  0202  2e2e  0000  0b00  0000  1400  0a02  ........
0000020:   6c6f  7374  2b66  6f75  6e64  0000  0c00  0000  lost+found.....
0000030:   1400  0c01  2e77  682e  2e77  682e  6175  6673  ......wh..wh.aufs
0000040:   011e  0000  1400  0c02  2e77  682e  2e77  682e  .........wh..wh.
0000050:   706c  6e6b  015a  0000  1400  0c02  2e77  682e  plnk.Z......wh.
0000060:   2e77  682e  2e74  6d70  021e  0000  0c00  0402  .wh..tmp.......
0000070:   726f  6673  025a  0000  0c00  0302  6574  6300  rofs.Z.....etc.
0000080:   045a  0000  1000  0502  6364  726f  6d00  0000  .Z...cdrom......
0000090:   031e  0000  0c00  0302  7661  7200  013c  0000  ......var..<..
```

Metadata Layer Tools

- **istat**

  - Display details of a meta-data structure (i.e. inode)

```
user@forensics:~$ istat ubnist1.casper-rw.gen3.aff 20
inode: 20 Allocated Group: 0
Generation Id: 96054594
uid / gid: 0 / 0
mode: rrw-r--r--size: 123600 num of links: 0
Inode Times:
Accessed: Tue Jan 6 10:59:33 2009
File Modified: Wed Jan 7 07:59:47 2009
Inode Modified: Wed Jan 7 07:59:47 2009
Deleted:Wed Dec 31 16:00:17 1969
```

Metadata Layer Tools

- **icat**

  - Output the contents of a file based on its meta data address (inode number)

```
user@forensics:~$ icat ubnist1.casper-rw.gen3.aff 20
:
:
```

- **ifind**

  - Find the meta-data structure that has allocated a given disk unit or file name.

Journal Tools

- **jls command**
  - lists items in the file system journal

```
user@forensics:~$ jls ubnist1.casper-
rw.gen3.E01
```

Image File Tools

- **img_stat**
  - display information about the image format, Including any hash information and other case-relevant metadata contained in the image

```
user@forensics:~$ img_stat ubnist1.casper-rw.gen3.aff IMAGE
FILE INFORMATION
--------------------------------------------
Image Type: AFF
Size in bytes: 629145600
MD5: 717f6be298748ee7d6ce3e4b9ed63459
SHA1: 61bcd153fc91e680791aa39455688eab946c4b7
Creator: afconvert
Image GID: 25817565F05DFD8CAEC5CFC6B1FAB45
Acquisition Date: 2009-01-28 20:39:30
AFFLib Version: "3.3.5"
```

Forensic Imaging

Bit-by-bit | Byte-for-byte | Looking at a mirror

Forensic Imaging

- generating an **exact duplicate** (or a container that holds an exact duplicate) of a source media

- acquiring a **complete** sector-for-sector, byte- for-byte copy of original media.

Needs to get even the deleted data and file slack

Forensic Imaging

dd

most basic open-source tool available to **copy a disk**

```
dd if=/dev/sda of=/dev/sdb bs=4096
```

source

destination

block size

Forensic Imaging

dd

most basic open-source tool available to **create a forensic image**

```
dd if=/dev/sda of=test.img  bs=4096
```

source

destination

block size

Forensic Imaging

dcfldd

- dd tool that is specifically designed for forensic use
- created for the Defense Computer Forensics Laboratory
- aside from features offered by dd, it can do:
  - hash creation and validation
  - logging of activity
  - splitting the output file into fixed-size chunks

```
dcfldd if=/dev/sda of=test.img bs=4096
```

Hashing

Baby hash hash

Hashing

Cryptographic hash function

- takes an arbitrary amount of data as input and returns a fixed-size string as output (Hash value)

Goal

- modifications of a single bit of input data will produce a radically different hash value

Application

- verification of the integrity of digital artifact

Hashing

Cryptographic hash function

- takes an arbitrary amount of data as input and returns a fixed-size string as output (Hash value)

Algorithms

- MD5 – 128 bit
- SHA1 – 160 bit
- SHA 256 – 256 bit
- SHA512 – 512 bits

Hashing

How many possible hash values does MD5 (128-bit) have?

2^{128}

Is it possible for two files to have the same MD5 hash value?

Yes

How many unique files are needed to ensure (100% assurance) that at least two files will have the same MD5 hash value?

$2^{128} + 1$

Hashing Tools

MD5deep

- Suite of hashing utilities designed to recurse through a set of input files or directories and produce hash lists for these
- Good with checking file integrity
- Not good if you want to prove that two files are similar but not identical

```
MD5 (lorem1.txt) = ea4884844ddb6cdc55aa7a95d19815a2
MD5 (lorem2.txt) = 9909552a79ed968a336ca3b9e96aca66
```

Hashing Tools

ssdeep

- context triggered piecewise hashes
- Uses fuzzy hashing

```
24:FPYOEMR7SlPYzvH6juMtTtqULiveqrTFIoCPddBjMxiAyejao: 9YfQ7
qYza6MdtiHrTKoCddBQxiwd,"/home/cory/ssdeeptest/lorem1.txt"
24:lPYOEMR7SlPYzvH6juMtTtqULiveqrTFIoCPddBjMxiAyejao: dYfQ7
qYza6MdtiHrTKoCddBQxiwd,"/home/cory/ssdeep-test/lorem2.txt"
```

Carving

Carving for some French fries and sweets

Carving

“When all else fails, we carve”

- Searching a data stream for file headers and magic values
- Determining (or guessing) the file end point, and saving this sub-stream out into a carved file
- In other words: **Putting a file back together**

Carving Tool

Foremost

file carving program uses defined headers, footers, and knowledge of the internal structures for supported file types to aid in carving

```
user@ubuntu:~/dfrws $ foremost -v -i dfrws-2006- challenge.raw
Foremost version 1.5.4 by Jesse Kornblum, Kris Kendall, and
Nick Mikus
Audit File
Foremost started at Sat Dec 10 21:51:55 2010
Invocation: foremost -v -i dfrws-2006-challenge.raw
Num  Name(bs=512)      Size File    Offset  Comment
0:    00003868.jpg     280  KB     1980416
1:    00008285.jpg     594  KB     4241920
2:    00011619.jpg     199  KB     5948928
3:    00012222.jpg     6    MB     6257664
```

Resources

ext-part-test-2.dd

https://dftt.sourceforge.net/test1/index.html

fat-img-kw.dd

https://dftt.sourceforge.net/test2/index.html

SleuthKit Website

https://www.sleuthkit.org/sleuthkit/man